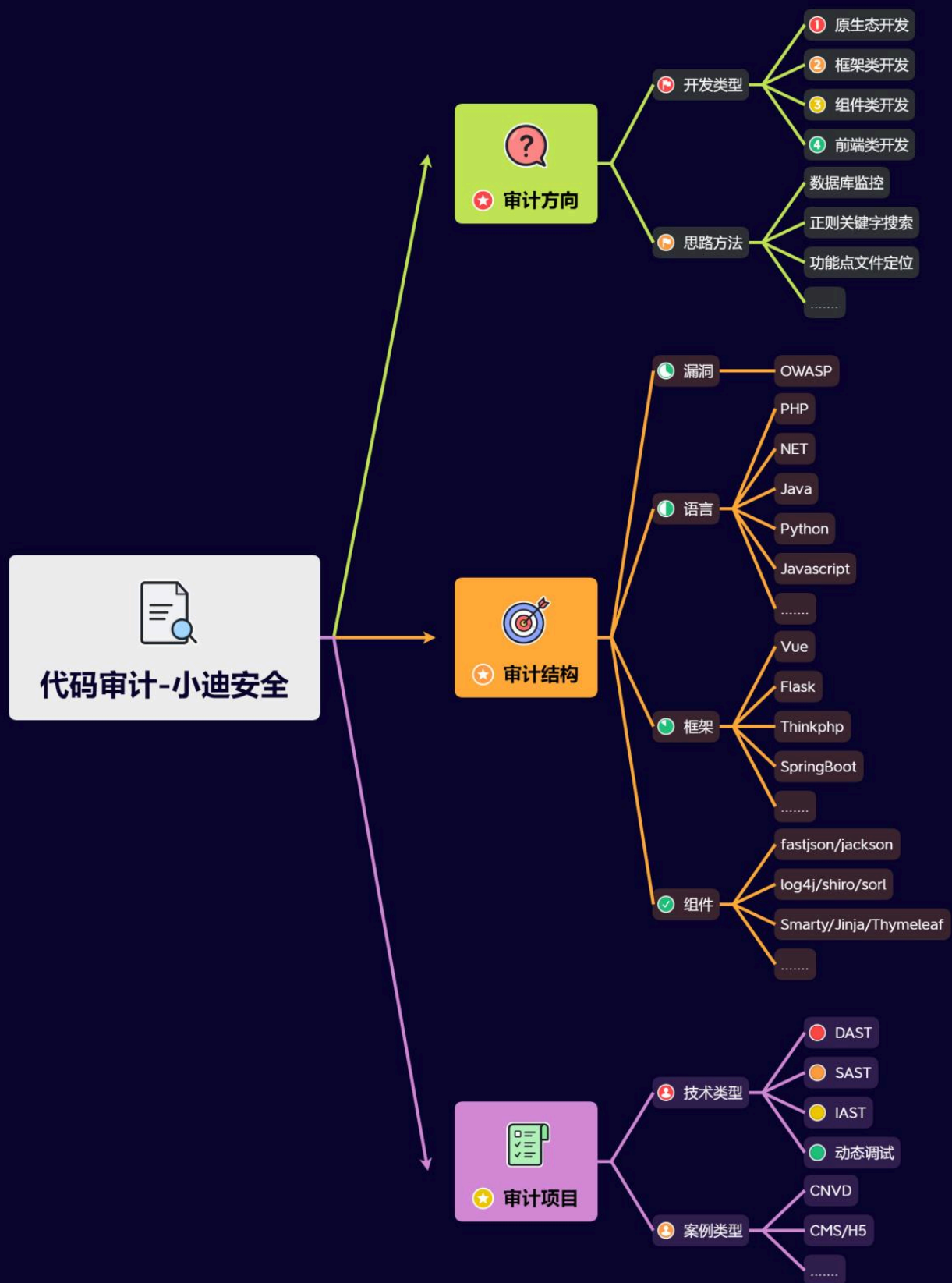


Day129 代码审计-SAST项目篇&CodeQL审计 &SCA插件&语言应用&开源商业&自动工具



1.知识点

- 1、PHP审计-原生态开发-SQL注入&语句监控
- 2、PHP审计-原生态开发-SQL注入&正则搜索
- 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-原生态开发-SQL注入&语句监控
 - 2、PHP审计-原生态开发-SQL注入&正则搜索
 - 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-MVC开发-RCE&代码执行
 - 2、PHP审计-MVC开发-RCE&命令执行
 - 3、PHP审计-MVC开发-RCE&文件对比

-
- 1、PHP审计-动态调试-未授权安全
 - 2、PHP审计-文件对比-未授权安全
 - 3、PHP审计-未授权访问-三种形态

-
- 1、PHP审计-动态调试-变量覆盖
 - 2、PHP审计-动态调试-原生反序列化
 - 3、PHP审计-动态调试-框架反序列化

-
- 1、PHP框架学习-ThinkPHP-架构&调试&路由&接受
 - 2、PHP框架审计-ThinkPHP-不安全写法&版本漏洞

-
- 1、PHP框架审计-ThinkPHP-反序列化链分析
 - 2、PHP框架审计-ThinkPHP-反序列化链挖掘

-
- 1、PHP框架审计-Yii-反序列化链分析
 - 2、PHP框架审计-Yii-实际CMS挖掘分析

-
- PHP代审项目-TP框架-实战某违法微盘

-
- 1、ASP.NET审计-SQL注入-继承架构&DLL反编译
 - 2、ASP.NET审计-SQL注入-数据库监控&关键字搜索

-
- 1、ASP.NET审计-文件上传-DLL反编译&关键字搜索
 - 2、ASP.NET审计-文件上传-预编译解析&关键字搜索
-

- 1、ASP.NET审计-未授权访问-未引用脆弱鉴权
 - 2、ASP.NET审计-未授权访问-不严谨鉴权逻辑
 - 3、ASP.NET审计-动态调试-附加进程&断点调试
 - 4、ASP.NET审计-程序逆向-EXE逻辑&修改覆盖
-

- 1、JavaEE审计-文件安全-上传&下载&读取&写入等
 - 2、JavaEE审计-文件安全-功能点追踪&关键字搜索
-

- 1、JavaEE审计-第三方组件-检测插件&利用条件
 - 2、JavaEE审计-第三方组件-FastJson&Log4j
 - 3、JavaEE审计-第三方组件-Shrio&H2database
-

- 1、JavaEE审计-SSTI注入-FreeMarker&Thymeleaf
 - 2、JavaEE审计-补充Checklist-功能点&搜索关键
-

- 1、JavaEE审计-FastJson-反序列化漏洞形成
 - 2、JavaEE审计-FastJson-低版本CC利用链分析
 - 3、JavaEE审计-FastJson-高版本CC利用链绕过
-

- 1、JavaEE审计-原生开发-反序列化
 - 2、JavaEE审计-Shiro-550反序列化
 - 3、JavaEE审计-Shiro-URLDNS利用链
-

- 1、JavaEE审计-Shiro-550反序列化
 - 2、JavaEE审计-Shiro-gadget&sink&source
 - 2、JavaEE审计-Shiro-CC2&CB1无依赖利用链
-

- 1、JavaEE审计-CB链-TemplatesImpl&字节码加载
 - 2、JavaEE审计-CC链-Transform&TransformedMap&LazyMap
-

- 1、JavaEE审计-JNDI注入-LDAP&RMI服务
- 2、JavaEE审计-JNDI注入-LDAP&RMI高版本
- 3、JavaEE审计-JNDI注入-LDAP&RMI反序列化

- 1、JavaEE审计-Tomcat-内存马分类&动态注册原理
- 2、JavaEE审计-Tomcat-Listen&Filter&Servlet

- 1、SAST-IDEA插件SCA-依赖组件漏洞-JAVA
- 2、SAST-Fortify&CheckMarx&CodeQL-各语言
- 3、SAST-Bandit&NodeJsScan-Python&NodeJS

2.详细点

对比项	DAST	SAST	IAST
测试对象	Web应用程序	Web应用程序 APP的漏洞	Web应用程序 APP的漏洞
部署成本	低	低	高
使用成本	较低, 基本无需人工验证	高, 人工排除误报	低, 基本没有误报
漏洞检出率	中	高	较高
脏数据	非常多	较少	几乎没有
研发流程集成	测试/线上运营阶段	研发阶段	测试阶段
误报率	低	高	极低 (几乎为0)
测试覆盖度	低	高	高
检查速度	随测试用例数量稳定增加	随代码量呈指数增长	实时检测
逻辑漏洞检测	支持部分	不支持	支持部分
影响漏洞检出率因素	与测试payload覆盖度相关 企业可优化和扩展	与检测策略相关 企业可在定制策略	与检测策略相关 企业可定制测量
第三方组件漏洞检测	支持	不支持	支持
支持语言	不区分语言	区分语言	区分语言
支持框架	不区分框架	区分框架	区分框架
侵入性	较高, 脏数据	低	低
风险程度	较高, 扫挂/脏数据	低	低
漏洞详情	中, 请求	较高, 数据流+代码行数	高, 请求+数据流+代码行数
CI/CD集成	不支持	支持	支持
持续安全测试	不支持	支持	支持
工具集成	无	开发环境集成 构建工具、问题跟踪工具	构建工具、自动化
其他	无法定位漏洞的具体代码行数和产生漏洞的原因		不支持C, C ++和Golang等语言

对比项	DAST	SAST	IAST
商业产品	AppScan、AWVS、webinspect burpsuite	Fortify、Checkmarx	默安-雳鉴IAST 新思Seeker软件 开源网安SecZone VulHunter 、墨云VackBot等，国外：Contrast Security等
开源产品	Owasp ZAP、Xray	Raptor、RIPS、Seay源代码审计系统、VCG等	百度RASP等
部署成本	低	低	高
使用成本	较低, 基本无需人工验证	高, 人工排除误报	低, 基本没有误报
漏洞检出率	中	高	较高

3.演示案例

3.1 SAST-IDEA插件SCA-依赖组件漏洞-JAVA

```

1  #SAST-IDEA插件SCA-依赖组件漏洞-JAVA
2  1、IDEA插件：（专业针对项目依赖组件审计插件）
3  -CodeArts Check
4  https://blog.csdn.net/hwxiaozhi/article/details/133674492
5  -MurphySec Code Scan
6  https://github.com/murphysecurity/murphysec-jetbrains-plugin
7  -OpenSCA-intellij-plugin
8  https://opensca.xmirror.cn/docs/v1/
9  2、商业平台：（综合SDL项目，不单是漏洞审计等）
10 TCA, OpenSCA, MurphySec, Huawei, Aliyun等

```

3.2 SAST-Fortify&CheckMarx&CodeQL-各语言

```

1  #SAST-Fortify&CheckMarx&CodeQL-各语言
2  优点：大部分主流语言都支持，使用简单
3  缺点：部分语言组件审计不支持，针对性差
4  因此诞生CodeQL项目，可玩性强，研究颇多
5  0、安装环境
6  -安装命令行codeql-cli(执行文件)添加到path目录，设置为全局变量。
7  https://github.com/github/codeql-cli-binaries/releases
8  -安装codeql查询引擎（SDK引擎）
9  https://github.com/github/codeql/releases
10 codeql-cli同目录下(因为codeql-cli把同层目录作为查找路径)
11 1、创建数据库
12 codeql database create tmall --language="java" --command="mvn clean install --file pom.xml -Dmaven.test.skip=true" --source-root=E:/code/java/129/Tmall_demo-master --overwrite

```

```
13 2、执行扫描规则
14 codeql database analyze tmall E:\code\java\129\codeql-codeql-cli-
    v2.15.4\java\ql\src\codeql-suites\java-code-scanning.qls --format=csv --
    output=result.csv
15
```

3.3 SAST-Bandit&NodeJsScan-Python&NodeJS

```
1 #SAST-Bandit&NodeJsScan-Python&NodeJS
2 1、Bandit（专业Python语言应用审计项目）
3 https://github.com/PyCQA/bandit
4 参考: https://bandit.readthedocs.io/
5 安装: pip install bandit
6 linux:
7 安装后会在当前Python目录下bin
8 使用: bandit -r 需要审计的源码目录
9 windows:
10 安装后会在当前Python目录下script
11 使用: bandit -r 需要审计的源码目录
12 2、NodeJsScan（专业NodeJS语言应用审计项目）
13 https://github.com/ajinabraham/NodeJsScan
14 docker pull opensecurity/nodejsscan:latest
15 docker run -it -p 9090:9090 opensecurity/nodejsscan:latest
```

资源:

```
1 https://github.com/github/codeql 神器
2 https://github.com/PyCQA/bandit Python
3 https://github.com/securego/gosec Go
4 https://github.com/LoRexxar/Kunlun-M PHP&JS
5 https://github.com/presidentbeef/brakeman Ruby
6 https://github.com/ajinabraham/NodeJsScan JS框架
```